Image Processing with OpenCV

CSE480/CSE482 Lab Assignment 01

Mechatronics Engineering and Automation Program

Student name _____ Student ID _____

Section _____ Submission date _____

Abstract

Four image-processing experiments were implemented in Python using OpenCV and NumPy: selective colour isolation, weighted image blending, vignette filtering, and channel swapping. A red-dominance mask preserved selected apple pixels against a grayscale background; weighted sums produced translucent overlays; a blurred circular mask darkened the rover image edges; and channel permutations altered the zebra image colours. All four tasks executed successfully, and the 14 saved output images matched a fresh execution exactly. The results demonstrate how channel values and masks control colour, transparency, and local brightness.

Introduction and objectives

A digital colour image stores separate intensity values for three channels. OpenCV reads colour images in blue, green, red order, commonly written as BGR. Manipulating these channels and combining image arrays provides a direct way to change image appearance. This laboratory investigates both discrete selection, using a binary mask, and continuous transformation, using fractional blending weights and a smooth spatial mask.

The objectives were to isolate a selected colour using thresholds and bitwise operations, blend equal-sized images at several transparency levels, create a smooth vignette using Gaussian blur, and observe the effect of rearranging colour channels. The procedures follow the four exercises in the assignment sheet [1].

Software and input images

The implementation is contained in lab01.py and uses OpenCV 5.0.0 and NumPy 2.5.4 in the supplied virtual environment. Images are loaded as three-channel colour arrays. The dimensions below are stated as width by height.

Image	Dimensions	Use
apple.png	300 × 300	Selective colour isolation
cameraman.jpg	225 × 225	Base image for blending
zebra.png	425 × 313	Overlay and channel swapping
marsrover.png	853 × 480	Vignette filtering

Task 1 Selective colour isolation

Method

The apple image was converted to grayscale and then back to three channels so that it could be combined with the original BGR image. The original image was split into blue, green, and red arrays. A pixel was selected only when its red intensity exceeded 130 and exceeded each other channel by more than 20 intensity levels.

```
M = 255 where R > 130, R > G + 20, and R > B + 20
M = 0 elsewhere
```

Channel differences were computed with signed 16-bit values to avoid unsigned overflow. The original colour pixels were extracted using cv2.bitwise_and with M. The grayscale background was extracted using the inverse mask, and cv2.bitwise_or combined the two regions.

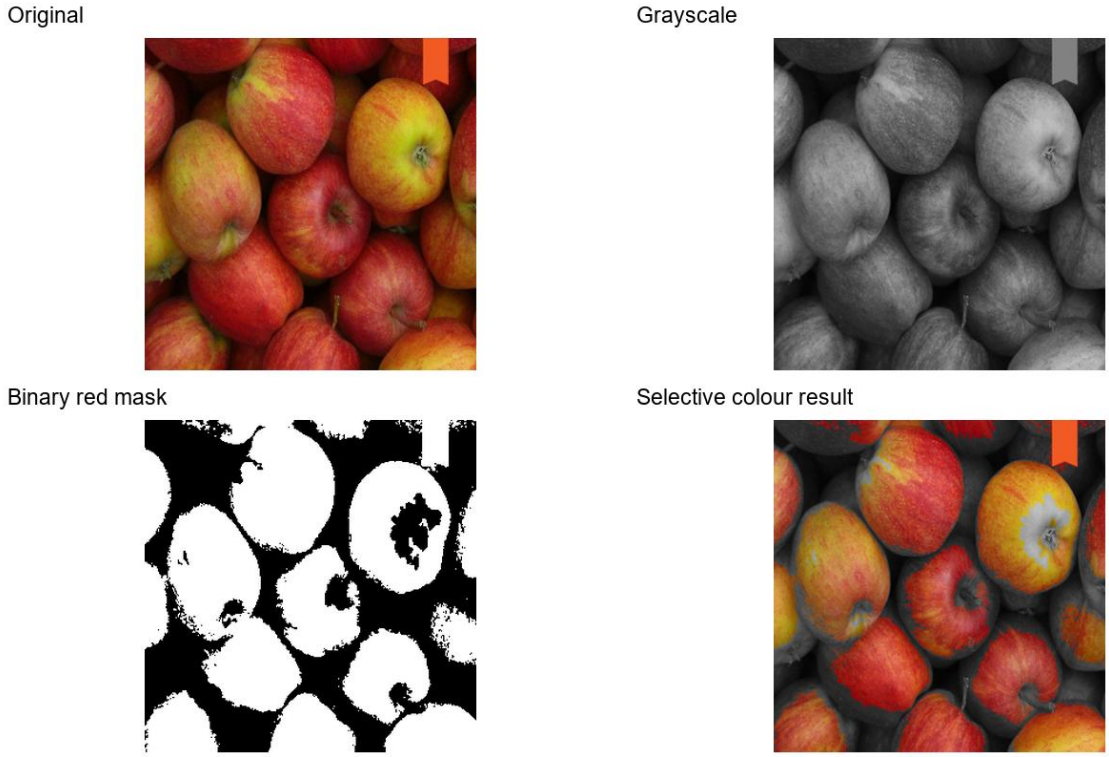


Figure 1. Original image, grayscale image, binary selection mask, and selective colour output.

Results and discussion

The selected apple regions retain their original colours, while unselected pixels have equal blue, green, and red values and therefore appear grayscale. The binary mask selects 58.39% of the image pixels. This percentage measures threshold selection rather than segmentation accuracy.

Some orange and yellow regions remain coloured because they also satisfy the red-dominance rule. The selection also affects small coloured background details. This behaviour follows the pixel-based criterion: the mask identifies colour values rather than one complete apple. Tighter thresholds could reduce spill but would also remove valid red pixels in shadows.

Task 2 Creating a ghostly overlay

Method

The cameraman image was used as the base. The zebra image was resized from 425 × 313 to 225 × 225 pixels using cv2.resize with INTER_AREA, matching the base image dimensions. Both arrays therefore had compatible sizes and channel counts for weighted addition.

```
Iout = alpha × Icameraman + beta × Izebra
beta = 1 - alpha
cv2.addWeighted(base, alpha, ghost, beta, 0)
```

Three alpha values were tested: 0.35, 0.50, and 0.65. The corresponding zebra weights were 0.65, 0.50, and 0.35. The additive brightness term was zero. Weights summing to one produce a convex combination of the two pixel values before output rounding.

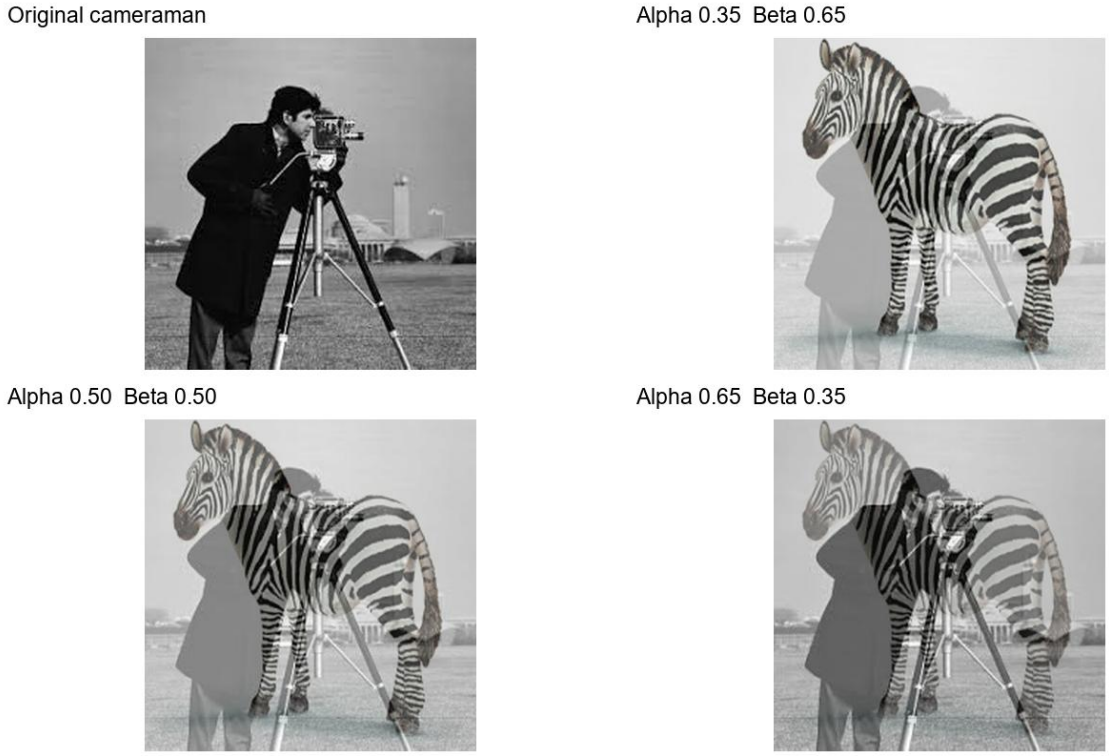


Figure 2. Cameraman base image and three overlays. Increasing alpha makes the zebra fainter.

Results and discussion

The zebra appears translucent over the cameraman. At alpha = 0.35, the zebra contributes most strongly; at alpha = 0.50, both images contribute equally; and at alpha = 0.65, the cameraman is more prominent. The effect demonstrates how a linear combination controls visual transparency.

The entire zebra image is blended, including its white background, which lightens parts of the cameraman scene. Resizing to a square also changes the zebra aspect ratio. These effects are acceptable for the specified blending exercise. Subject segmentation and aspect-ratio-preserving placement could improve a more realistic composite.

Task 3 Artistic vignette filtering

Method

A single-channel black mask was created with the same spatial dimensions as marsrover.png. A filled white circle was drawn at the image centre. Its radius was int(0.45 × min(height, width)), giving 216 pixels. Gaussian blur with sigma = min(height, width)/8 = 60 pixels softened the circle boundary; a kernel size of (0, 0) allowed OpenCV to choose the kernel from sigma.

```
W = blurred_mask / max(blurred_mask)
Iout(x, y, c) = Ioriginal(x, y, c) × W(x, y)
```

The blurred mask was converted to floating point and normalized by its maximum. Each BGR channel was multiplied by the same spatial weights. Values were clipped to the range 0 to 255 and converted back to unsigned 8-bit integers.

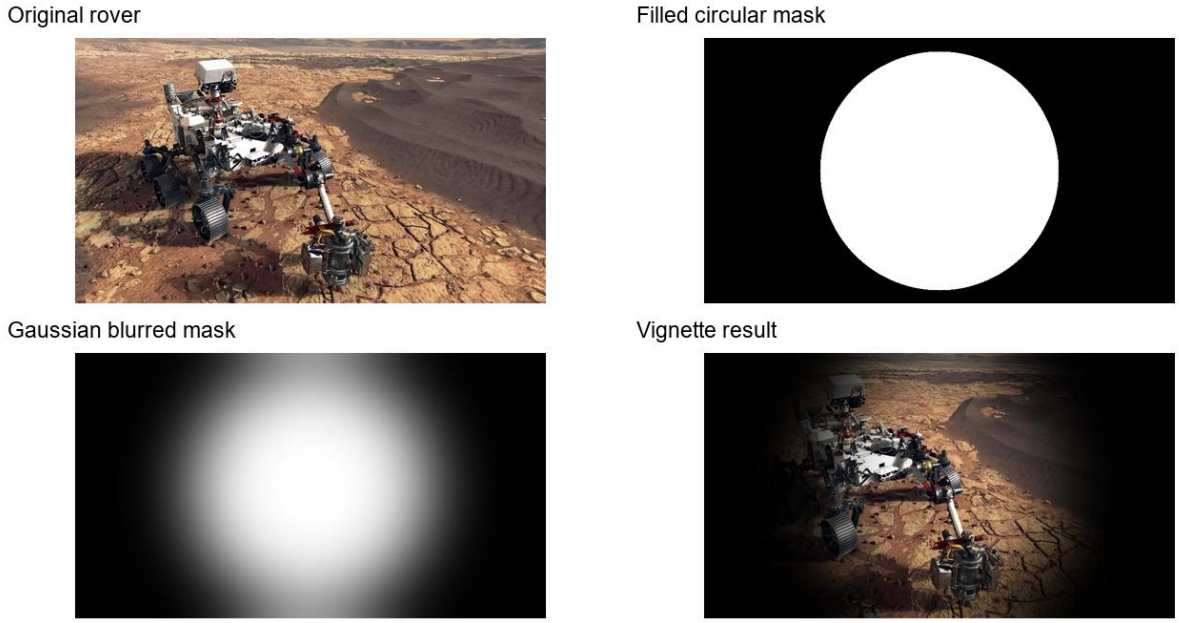


Figure 3. Original rover image, circular mask, blurred mask, and final vignette.

Results and discussion

The vignette smoothly darkens the outer image and directs attention toward the central region. The saved blurred mask contains all 256 possible 8-bit intensity levels, confirming a graded transition rather than a binary boundary. Normalizing by the maximum preserves full brightness at the strongest mask location.

Multiplication is essential to the smooth effect. A mask argument in a bitwise operation selects pixels whenever its value is nonzero; it does not apply fractional brightness weights. The circular mask also creates pronounced darkening along the wider sides of this landscape image. An elliptical mask could retain more of the horizontal scene if a gentler effect were desired.

Task 4 Channel swapping

Method

The zebra image was separated into its original blue, green, and red channels using cv2.split. Three alternative sequences were passed to cv2.merge. Since OpenCV interprets the merged output as BGR, the last supplied array becomes the displayed red channel.

Output	Merge sequence	Channel mapping
Green and red swap	(B, R, G)	B stays blue; R becomes green; G becomes red
Blue and red swap	(R, G, B)	R becomes blue; G stays green; B becomes red
Channel rotation	(G, R, B)	G becomes blue; R becomes green; B becomes red

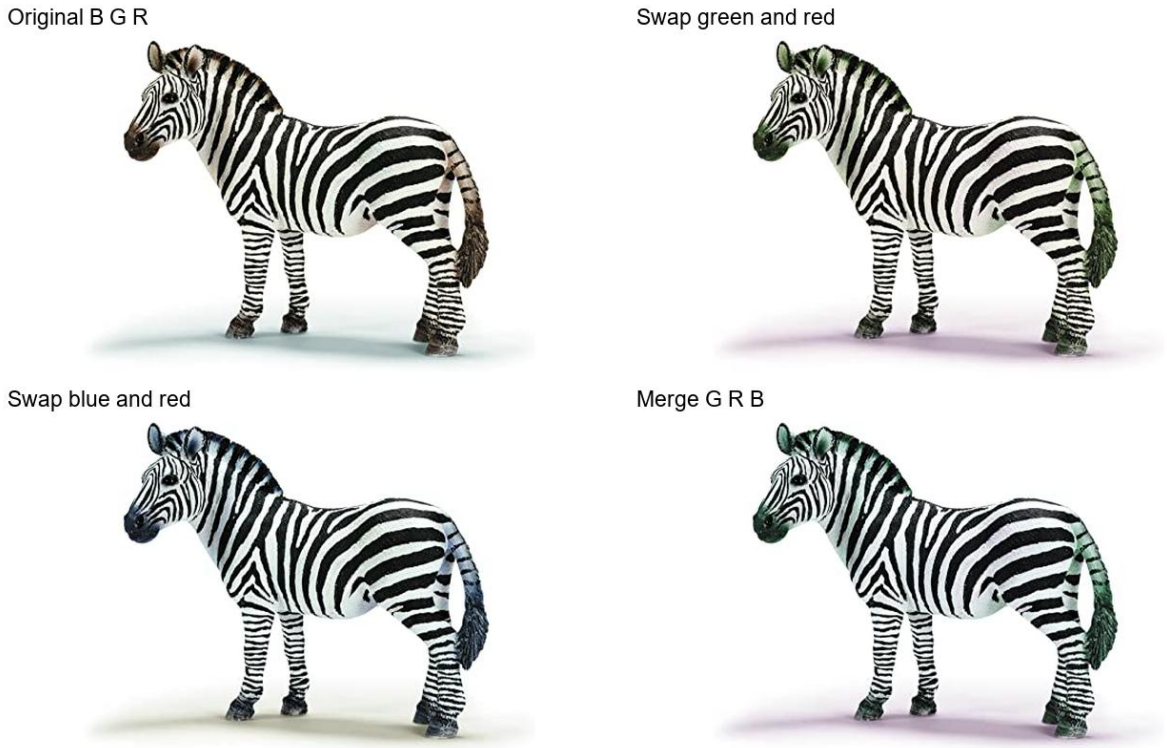


Figure 4. Original zebra and three channel permutations. Differences are strongest in tinted regions.

Results and discussion

The green and red swap changes the zebra highlights and surrounding shadow toward green and purple tones. Other permutations redistribute the original tints differently. The image structure and spatial detail remain fixed because the operation rearranges channel values without moving pixels.

The effect is subtle because much of the supplied zebra image is black, white, or nearly neutral. Where B = G = R, every channel permutation produces the same pixel value. Strongly coloured input would show a more dramatic surreal transformation. The supplied image still demonstrates the required channel mapping, including original green values appearing in the output red channel.

Verification and reproducibility

All four tasks were executed using the prepared local virtual environment. A separate verification run wrote its results to a temporary review directory. All 14 PNG outputs, including masks, blend variations, channel permutations, and the comparison panel, matched the existing saved images exactly at the pixel level.

Additional checks confirmed that the Task 1 mask contains only 0 and 255, and that all unselected pixels in the selective-colour result are grayscale. Inspection of the Task 3 mask confirmed 256 distinct intensity levels. The comparisons establish reproducibility of this implementation; they do not constitute a general segmentation-accuracy or perceptual-quality benchmark.

Running the experiment

From the project folder, run the following command in PowerShell. The input images are read from Test Images/Test Images, and the results are saved in outputs.

```
.\.venv\Scripts\python.exe .\lab01.py
```

An individual exercise can be selected with --task 1, --task 2, --task 3, or --task 4. The --input-dir and --output-dir options allow different image and result directories. The requirements.txt file lists numpy and opencv-python. The original input images are not modified.

Conclusion

The four experiments met the specified processing requirements. Binary masks combined original colour with a grayscale background, weighted addition controlled the relative visibility of two images, a Gaussian-blurred mask produced continuous edge darkening, and channel permutations changed colour interpretation while retaining image geometry.

The main practical lesson is that the numerical meaning of an operation determines its visual result. Colour thresholds do not identify object boundaries, whole-image blending includes the source background, fractional masks require arithmetic scaling, and channel swapping has little visible effect on neutral pixels. These observations explain both the successful effects and the limitations visible in the results.

References

[1] Mechatronics Engineering and Automation Program. CSE480/CSE482 Lab Assignment #01. Supplied assignment sheet, lab01_tasks (1).pdf.

[2] Laboratory implementation and execution materials. lab01.py, README.md, requirements.txt, supplied Test Images, and generated outputs in the project folder.